ADVANCED PYTHON PROGRAMMING

ASSESSMENT 1

Name:VIKNESH.G
Reg No:23MID0161

TASK 1

Chinese Text Encoding Conversion

In [1]:
```python
# Original Chinese text
chinese_text = "你好, 世界"

print("Original Text:")
print(chinese_text)

# Step 1: Encode to GBK (a Chinese-specific encoding, preserves the characters)
gbk_bytes = chinese_text.encode("gbk")
print("\nGBK Encoded Bytes:")
print(gbk_bytes)

# Step 2: Decode back from GBK
gbk_text = gbk_bytes.decode("gbk")
print("\nAfter GBK Decoding:")
print(gbk_text)

# Step 3: Encode the text into UTF-8 (the standard universal encoding)
utf8_bytes = chinese_text.encode("utf-8")
print("\nUTF-8 Encoded Bytes:")
print(utf8_bytes)

# Step 4: Decode back from UTF-8
final_text = utf8_bytes.decode("utf-8")
```

```
print("\nFinal Decoded Text:")
print(final_text)
```

```
Original Text:
你好，世界

GBK Encoded Bytes:
b'\xc4\xe3\xba\xc3, \xca\xc0\xbd\xe7'

After GBK Decoding:
你好，世界

UTF-8 Encoded Bytes:
b'\xe4\xbd\xa0\xe5\xa5\xbd, \xe4\xb8\x96\xe7\x95\x8c'

Final Decoded Text:
你好，世界
```

TASK 2

File Formats

2.1 PYPDF

Feature 1: Count Total Number of Pages

In [18]:
```
!pip install pypdf
from pypdf import PdfReader

reader = PdfReader("sample.pdf")   # replace with your own PDF file name
print("Total Number of Pages:", len(reader.pages))
```

```
Collecting pypdf
  Downloading pypdf-6.14.2-py3-none-any.whl.metadata (7.2 kB)
Downloading pypdf-6.14.2-py3-none-any.whl (349 kB)
Installing collected packages: pypdf
Successfully installed pypdf-6.14.2
Total Number of Pages: 3
```

Feature 2: Extract Text from PDF

In [19]:
```python
from pypdf import PdfReader

reader = PdfReader("sample.pdf")

for i, page in enumerate(reader.pages):
    print(f"\n----- Page {i+1} -----")
    print(page.extract_text())
```

```
----- Page 1 -----
RESEARCH PAPER BT
Abstract
This paper discusses a scalable architecture
for research data analysis using Python.
1. Introduction
Research into automated data processing has
grown rapidly in recent years.


----- Page 2 -----
2. Related Work
Several methods have been proposed in the
research literature for automating this process.
3. Methodology
We describe the methodology used to conduct
this research study.


----- Page 3 -----
4. Results
The results of our experiments show significant
improvement over baseline approaches.
5. Discussion
This section discusses the implications of
our research findings.
```

Feature 3: Search for a Keyword in the PDF

In [20]:
```python
from pypdf import PdfReader

reader = PdfReader("sample.pdf")

keyword = input("Enter keyword to search: ")
found = False

for i, page in enumerate(reader.pages):
    text = page.extract_text()
    if text and keyword.lower() in text.lower():
        print(f"Keyword found on Page {i+1}")
        found = True

if not found:
    print("Keyword not found.")
```

```
Keyword found on Page 1
```

Feature 4: Rotate the First Page by 90°

In [21]:
```python
from pypdf import PdfReader, PdfWriter

reader = PdfReader("sample.pdf")
writer = PdfWriter()

page = reader.pages[0]
page.rotate(90)
writer.add_page(page)

for i in range(1, len(reader.pages)):
    writer.add_page(reader.pages[i])

with open("Rotated_sample.pdf", "wb") as output:
    writer.write(output)

print("Rotated PDF saved successfully.")
```

```
Rotated PDF saved successfully.
```

2.2 JSON File Handling

Task 1: Load JSON file & audit structure

In [22]:
```
import json
import os

json_file_path = "input_data.json"

if not os.path.exists(json_file_path):
    print(f"Error: '{json_file_path}' not found.")
else:
    with open(json_file_path, "r", encoding="utf-8") as file:
        data = json.load(file)

    print("Successfully parsed file structure.")
    if isinstance(data, dict):
        print(f"Root Type: Dictionary | Top-Level Keys Found: {list(data.keys())}")
    elif isinstance(data, list):
        print(f"Root Type: List Array | Total Elements Identified: {len(data)}")
    print("-" * 50)
```

```
Successfully parsed file structure.
Root Type: Dictionary | Top-Level Keys Found: ['dashboard_name', 'last_updated', 'refresh_rate_seconds', 'system_node
s', 'global_configuration']
--------------------------------------------------
```

Task 2: Scan for missing / null values

In [23]:
```
null_counter = 0

def scan_for_nulls(element):
    global null_counter
    if element is None:
        null_counter += 1
    elif isinstance(element, dict):
        for value in element.values():
            scan_for_nulls(value)
    elif isinstance(element, list):
        for item in element:
            scan_for_nulls(item)

scan_for_nulls(data)
```

```
print(f"Audit Complete: Found {null_counter} empty (null/None) field values.")
print("-" * 50)
```

```
Audit Complete: Found 2 empty (null/None) field values.
--------------------------------------------------
```

Task 3: Search for a keyword across all keys/values

In [24]:
```
search_keyword = "Active"   # change this to whatever you want to search for
match_count = 0

def search_nodes(element, query):
    global match_count
    if isinstance(element, dict):
        for key, val in element.items():
            if query.lower() in str(key).lower() or query.lower() in str(val).lower():
                print(f" -> Found Match in Block Key [{key}]: {val}")
                match_count += 1
            search_nodes(val, query)
    elif isinstance(element, list):
        for item in element:
            search_nodes(item, query)

print(f"Searching document logs for instances containing: '{search_keyword}'")
search_nodes(data, search_keyword)
print(f"Search Results: Identified {match_count} exact text match vectors.")
print("-" * 50)
```

```
Searching document logs for instances containing: 'Active'
 -> Found Match in Block Key [system_nodes]: [{'node_id': 'SRV-001', 'host_name': 'primary-auth-cluster', 'status':
'Active', 'metrics': {'cpu_utilization_pct': 42.5, 'memory_available_gb': 12.8, 'active_connections': 1420}, 'recent_
alerts': []}, {'node_id': 'SRV-002', 'host_name': 'backup-ledger-db', 'status': 'Maintenance', 'metrics': {'cpu_utili
zation_pct': 88.1, 'memory_available_gb': 2.1, 'active_connections': 310}, 'recent_alerts': [{'alert_id': 'ALT-992',
'severity': 'Warning', 'message': 'High memory consumption threshold breached'}]}, {'node_id': 'SRV-003', 'host_nam
e': 'edge-load-balancer', 'status': 'Active', 'metrics': {'cpu_utilization_pct': 18.9, 'memory_available_gb': None,
'active_connections': 4890}, 'recent_alerts': []}]
 -> Found Match in Block Key [status]: Active
 -> Found Match in Block Key [metrics]: {'cpu_utilization_pct': 42.5, 'memory_available_gb': 12.8, 'active_connection
s': 1420}
 -> Found Match in Block Key [active_connections]: 1420
 -> Found Match in Block Key [metrics]: {'cpu_utilization_pct': 88.1, 'memory_available_gb': 2.1, 'active_connection
s': 310}
 -> Found Match in Block Key [active_connections]: 310
 -> Found Match in Block Key [status]: Active
 -> Found Match in Block Key [metrics]: {'cpu_utilization_pct': 18.9, 'memory_available_gb': None, 'active_connection
s': 4890}
 -> Found Match in Block Key [active_connections]: 4890
Search Results: Identified 9 exact text match vectors.
--------------------------------------------------
```

Task 4: Export a summary report as a new JSON file

In [25]:

```python
output_report_path = "json_processing_summary.json"

execution_summary = {
    "source_file": json_file_path,
    "structural_integrity": "Valid JSON Syntax Passed",
    "total_null_values_flagged": null_counter,
    "last_searched_term": search_keyword,
    "search_matches_extracted": match_count
}

with open(output_report_path, "w", encoding="utf-8") as out_file:
    json.dump(execution_summary, out_file, indent=4, ensure_ascii=False)

print(f"Success! Runtime processing logs exported cleanly to '{output_report_path}'.")
```

```
Success! Runtime processing logs exported cleanly to 'json_processing_summary.json'.
```

TASK 3

PANDAS

Feature 1: Display Dataset Information

In [26]:
```
import pandas as pd

df = pd.read_csv("test.csv")   # replace with your own CSV file
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 320 entries, 0 to 319
Data columns (total 9 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   year                  320 non-null    int64  
 1   industry_code_ANZSIC  320 non-null    object 
 2   industry_name_ANZSIC  320 non-null    object 
 3   rme_size_grp          320 non-null    object 
 4   variable              320 non-null    object 
 5   value                 320 non-null    int64  
 6   unit                  320 non-null    object 
 7   Unnamed: 7            0 non-null      float64
 8   Unnamed: 8            0 non-null      float64
dtypes: float64(2), int64(2), object(5)
memory usage: 22.6+ KB
None
```

Feature 2: Display Summary Statistics

In [27]:
```
import pandas as pd

df = pd.read_csv("test.csv")
print(df.describe())
```

	year	value	Unnamed: 7	Unnamed: 8
count	320.000000	320.000000	0.0	0.0
mean	2016.250000	25053.315625	NaN	NaN
std	4.152274	14235.381896	NaN	NaN
min	2011.000000	100.000000	NaN	NaN
25%	2013.250000	11787.750000	NaN	NaN
50%	2016.000000	26088.000000	NaN	NaN
75%	2019.000000	37445.500000	NaN	NaN
max	2022.000000	49993.000000	NaN	NaN

Feature 3: Count Missing Values

In [28]:
```
import pandas as pd

df = pd.read_csv("test.csv")

print("Missing Values in Each Column:")
print(df.isnull().sum())
```

```
Missing Values in Each Column:
year                      0
industry_code_ANZSIC      0
industry_name_ANZSIC      0
rme_size_grp              0
variable                  0
value                     0
unit                      0
Unnamed: 7              320
Unnamed: 8              320
dtype: int64
```

Feature 4: Filter Records by Year

In [31]:
```
import pandas as pd

df = pd.read_csv("test.csv")

year = int(input("Enter Year: "))
filtered = df[df["year"] == year]

print(filtered)
```

```
Empty DataFrame
Columns: [year, industry_code_ANZSIC, industry_name_ANZSIC, rme_size_grp, variable, value, unit, Unnamed: 7, Unnamed:
8]
Index: []
```

TASK 4

Audio Format Conversion

In [34]:
```
!pip install librosa soundfile
import librosa
import numpy as np
from scipy.signal import butter, filtfilt

# Load audio file
audio, sr = librosa.load("audio.wav", sr=None)   # replace with your own WAV file

print("========== AUDIO INFORMATION ==========")

# Feature 1: Sample Rate and Duration
duration = librosa.get_duration(y=audio, sr=sr)
print("Sample Rate:", sr, "Hz")
print("Duration:", round(duration, 2), "seconds")

# Feature 2: Peak Amplitude
peak_amplitude = np.max(np.abs(audio))
print("Peak Amplitude:", peak_amplitude)

# Feature 3: Dominant Frequency
fft = np.fft.fft(audio)
frequencies = np.fft.fftfreq(len(fft), d=1/sr)

dominant_frequency = abs(frequencies[np.argmax(np.abs(fft))])
print("Dominant Frequency:", round(dominant_frequency, 2), "Hz")

# Feature 4: Low-Pass Filter
cutoff = 3000              # Cutoff frequency in Hz
nyquist = sr / 2
normal_cutoff = cutoff / nyquist
```

```
b, a = butter(4, normal_cutoff, btype='low')
filtered_audio = filtfilt(b, a, audio)

print("Low-Pass Filter Applied Successfully!")
```

```
Collecting librosa
  Downloading librosa-0.11.0-py3-none-any.whl.metadata (8.7 kB)
Collecting soundfile
  Downloading soundfile-0.14.0-py2.py3-none-win_amd64.whl.metadata (18 kB)
Collecting audioread>=2.1.9 (from librosa)
  Downloading audioread-3.1.0-py3-none-any.whl.metadata (9.0 kB)
Requirement already satisfied: numba>=0.51.0 in c:\users\viknesh g\anaconda3\lib\site-packages (from librosa) (0.62.
1)
Requirement already satisfied: numpy>=1.22.3 in c:\users\viknesh g\anaconda3\lib\site-packages (from librosa) (2.3.5)
Requirement already satisfied: scipy>=1.6.0 in c:\users\viknesh g\anaconda3\lib\site-packages (from librosa) (1.16.3)
Requirement already satisfied: scikit-learn>=1.1.0 in c:\users\viknesh g\anaconda3\lib\site-packages (from librosa)
(1.7.2)
Requirement already satisfied: joblib>=1.0 in c:\users\viknesh g\anaconda3\lib\site-packages (from librosa) (1.5.2)
Requirement already satisfied: decorator>=4.3.0 in c:\users\viknesh g\anaconda3\lib\site-packages (from librosa) (5.
2.1)
Collecting pooch>=1.1 (from librosa)
  Downloading pooch-1.9.0-py3-none-any.whl.metadata (10 kB)
Collecting soxr>=0.3.2 (from librosa)
  Downloading soxr-1.1.0-cp312-abi3-win_amd64.whl.metadata (5.8 kB)
Requirement already satisfied: typing_extensions>=4.1.1 in c:\users\viknesh g\anaconda3\lib\site-packages (from libro
sa) (4.15.0)
Requirement already satisfied: lazy_loader>=0.1 in c:\users\viknesh g\anaconda3\lib\site-packages (from librosa) (0.
4)
Requirement already satisfied: msgpack>=1.0 in c:\users\viknesh g\anaconda3\lib\site-packages (from librosa) (1.1.1)
Collecting standard-aifc (from librosa)
  Downloading standard_aifc-3.13.0-py3-none-any.whl.metadata (969 bytes)
Collecting standard-sunau (from librosa)
  Downloading standard_sunau-3.13.0-py3-none-any.whl.metadata (914 bytes)
Requirement already satisfied: cffi>=1.0 in c:\users\viknesh g\anaconda3\lib\site-packages (from soundfile) (2.0.0)
Requirement already satisfied: pycparser in c:\users\viknesh g\anaconda3\lib\site-packages (from cffi>=1.0->soundfil
e) (2.23)
Requirement already satisfied: packaging in c:\users\viknesh g\anaconda3\lib\site-packages (from lazy_loader>=0.1->li
brosa) (25.0)
Requirement already satisfied: llvmlite<0.46,>=0.45.0dev0 in c:\users\viknesh g\anaconda3\lib\site-packages (from num
ba>=0.51.0->librosa) (0.45.1)
Requirement already satisfied: platformdirs>=2.5.0 in c:\users\viknesh g\anaconda3\lib\site-packages (from pooch>=1.1
->librosa) (4.5.0)
Requirement already satisfied: requests>=2.19.0 in c:\users\viknesh g\anaconda3\lib\site-packages (from pooch>=1.1->l
ibrosa) (2.32.5)
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\viknesh g\anaconda3\lib\site-packages (from reque
sts>=2.19.0->pooch>=1.1->librosa) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in c:\users\viknesh g\anaconda3\lib\site-packages (from requests>=2.19.0-
```

```
>pooch>=1.1->librosa) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\viknesh g\anaconda3\lib\site-packages (from requests>=
2.19.0->pooch>=1.1->librosa) (2.7.0)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\viknesh g\anaconda3\lib\site-packages (from requests>=
2.19.0->pooch>=1.1->librosa) (2025.11.12)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\viknesh g\anaconda3\lib\site-packages (from scikit-le
arn>=1.1.0->librosa) (3.5.0)
Collecting standard-chunk (from standard-aifc->librosa)
  Downloading standard_chunk-3.13.0-py3-none-any.whl.metadata (860 bytes)
Collecting audioop-lts (from standard-aifc->librosa)
  Downloading audioop_lts-0.2.2-cp313-abi3-win_amd64.whl.metadata (2.0 kB)
Downloading librosa-0.11.0-py3-none-any.whl (260 kB)
Downloading soundfile-0.14.0-py2.py3-none-win_amd64.whl (1.0 MB)
   ---------------------------------------- 0.0/1.0 MB ? eta -:--:--
   ---------------------------------------- 0.0/1.0 MB ? eta -:--:--
   ------------------------------ --------- 0.8/1.0 MB 5.3 MB/s eta 0:00:01
   ---------------------------------------- 1.0/1.0 MB 4.4 MB/s  0:00:00
Downloading audioread-3.1.0-py3-none-any.whl (23 kB)
Downloading pooch-1.9.0-py3-none-any.whl (67 kB)
Downloading soxr-1.1.0-cp312-abi3-win_amd64.whl (174 kB)
Downloading standard_aifc-3.13.0-py3-none-any.whl (10 kB)
Downloading audioop_lts-0.2.2-cp313-abi3-win_amd64.whl (30 kB)
Downloading standard_chunk-3.13.0-py3-none-any.whl (4.9 kB)
Downloading standard_sunau-3.13.0-py3-none-any.whl (7.4 kB)
Installing collected packages: standard-chunk, soxr, audioop-lts, standard-sunau, standard-aifc, soundfile, pooch, au
dioread, librosa

   ------------- -------------------------- 3/9 [standard-sunau]
   ------------------------- ------------- 6/9 [pooch]
   ------------------------------ -------- 7/9 [audioread]
   ---------------------------------- ---- 8/9 [librosa]
   ---------------------------------- ---- 8/9 [librosa]
   ---------------------------------------- 9/9 [librosa]

Successfully installed audioop-lts-0.2.2 audioread-3.1.0 librosa-0.11.0 pooch-1.9.0 soundfile-0.14.0 soxr-1.1.0 stand
ard-aifc-3.13.0 standard-chunk-3.13.0 standard-sunau-3.13.0
========== AUDIO INFORMATION ==========
Sample Rate: 44100 Hz
Duration: 5.0 seconds
Peak Amplitude: 0.6072998
Dominant Frequency: 440.0 Hz
Low-Pass Filter Applied Successfully!
```

Optional: Save the filtered audio to a new WAV file

In [35]:
```
import soundfile as sf

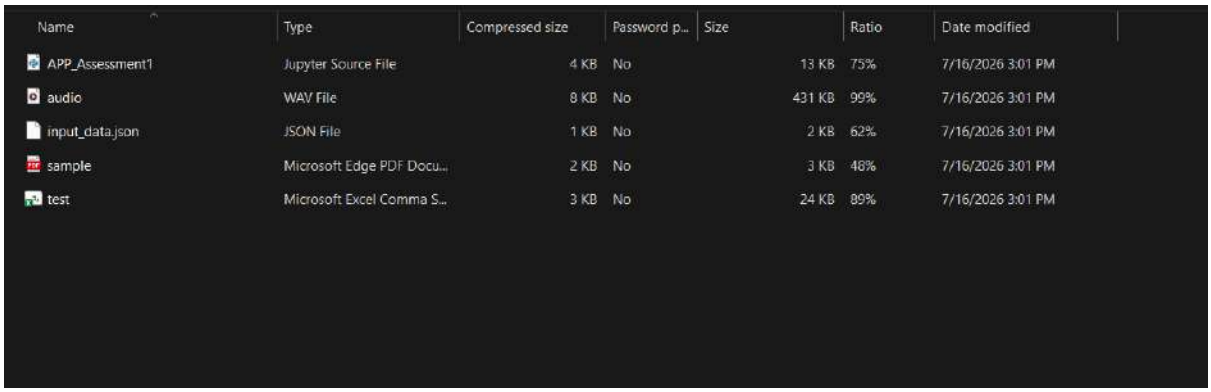
sf.write("filtered_audio.wav", filtered_audio, sr)
print("Filtered audio saved as 'filtered_audio.wav'")
```

```
Filtered audio saved as 'filtered_audio.wav'
```

In [ ]:
```

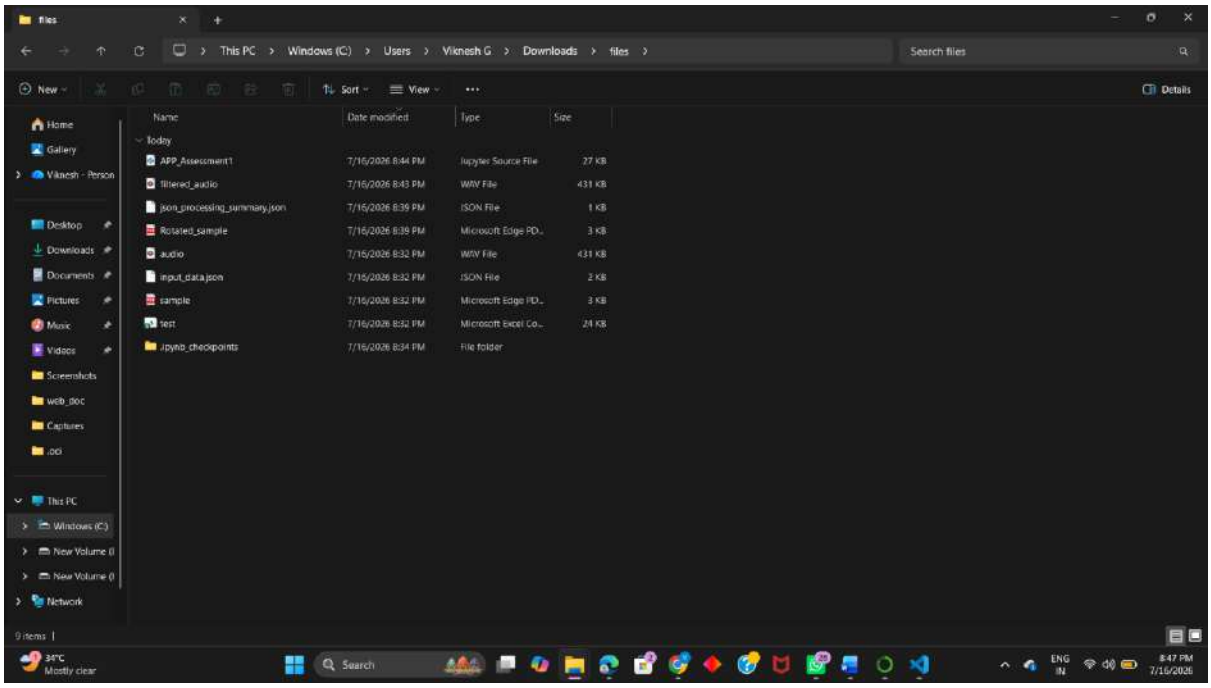
```

ATTACHMENT FILE :



Name	Type	Compressed size	Password p...	Size	Ratio	Date modified
APP_Assessment1	Jupyter Source File	4 KB	No	13 KB	75%	7/16/2026 3:01 PM
audio	WAV File	8 KB	No	431 KB	99%	7/16/2026 3:01 PM
input_data.json	JSON File	1 KB	No	2 KB	62%	7/16/2026 3:01 PM
sample	Microsoft Edge PDF Docu...	2 KB	No	3 KB	48%	7/16/2026 3:01 PM
test	Microsoft Excel Comma S...	3 KB	No	24 KB	89%	7/16/2026 3:01 PM

AFTER STORING THE NEW FILES:



JSON FILE DATA:

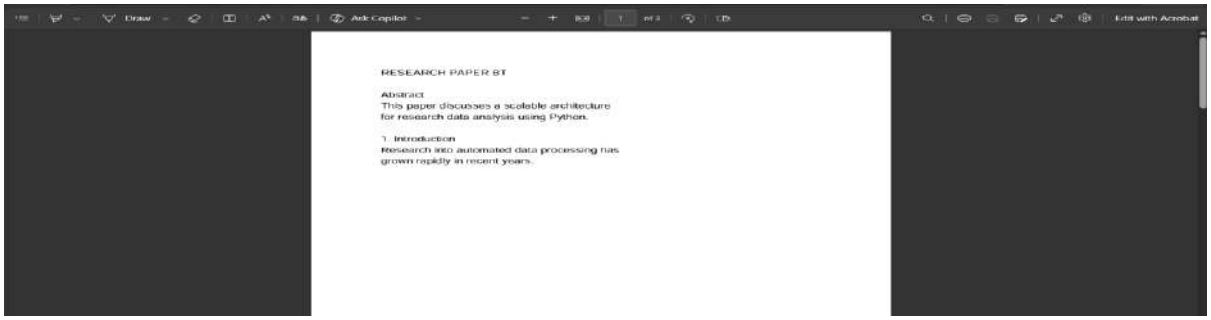
(SYNTESISED DATA)

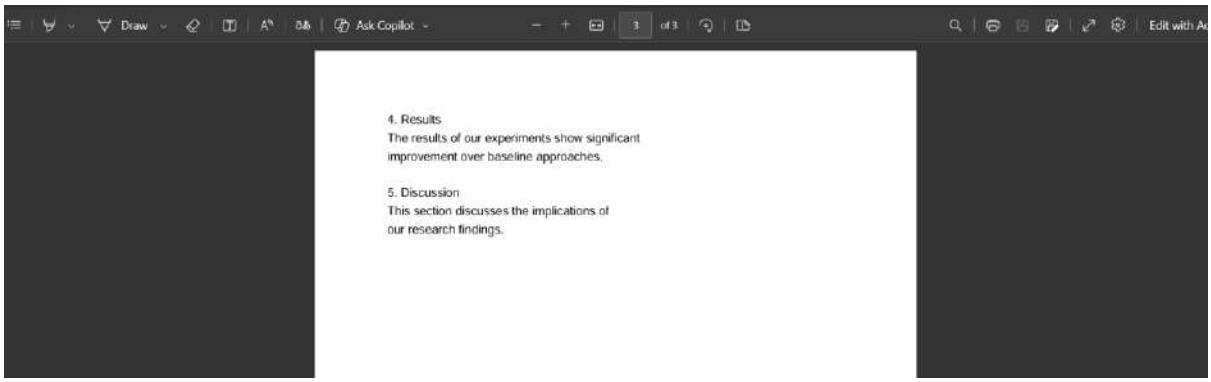

```
{
 "dashboard_name": "Server Monitoring Dashboard",
 "last_updated": "2026-07-10T10:00:00Z",
 "refresh_rate_seconds": 30,
 "system_nodes": [
  {
   "node_id": "SRV-001",
   "host_name": "primary-auth-cluster",
   "status": "Active",
   "metrics": {
```

```
      "cpu_utilization_pct": 42.5,
      "memory_available_gb": 12.8,
      "active_connections": 1420
    },
    "recent_alerts": []
  },
  {
    "node_id": "SRV-002",
    "host_name": "backup-ledger-db",
    "status": "Maintenance",
    "metrics": {
      "cpu_utilization_pct": 88.1,
      "memory_available_gb": 2.1,
      "active_connections": 310
    },
    "recent_alerts": [
      {
        "alert_id": "ALT-992",
        "severity": "Warning",
        "message": "High memory consumption threshold breached"
      }
    ]
  },
  {
    "node_id": "SRV-003",
    "host_name": "edge-load-balancer",
    "status": "Active",
    "metrics": {
      "cpu_utilization_pct": 18.9,
      "memory_available_gb": null,
      "active_connections": 4890
    },
    "recent_alerts": []
  }
],
"global_configuration": {
```

```
  "timezone": "UTC",
  "backup_enabled": true,
  "contact_email": null
 }
}
```


SAMPLE PDF:







CSV FILE SCREENSHOT:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	year	industry_c	industry_n	rme_size_g	variable	value	unit	Unnamed:	Unnamed: 8					
2	2011	A	Agriculture	a_0	Activity uni	19199	COUNT							
3	2011	A	Agriculture	a_0	Sales, gove	3066	COUNT							
4	2011	A	Agriculture	a_0	Total incon	34453	DOLLARS(millions)							
5	2011	A	Agriculture	a_0	Total exper	11094	DOLLARS(millions)							
6	2011	A	Agriculture	a_0	Total asset	38681	DOLLARS(millions)							
7	2011	A	Agriculture	b_0	Activity uni	44695	COUNT							
8	2011	A	Agriculture	b_0	Sales, gove	34859	COUNT							
9	2011	A	Agriculture	b_0	Total incon	11245	DOLLARS(millions)							
10	2011	A	Agriculture	b_0	Total exper	46683	DOLLARS(millions)							
11	2011	A	Agriculture	b_0	Total asset	39920	DOLLARS(millions)							
12	2011	A	Agriculture	c_0	Activity uni	7951	COUNT							
13	2011	A	Agriculture	c_0	Sales, gove	17051	COUNT							
14	2011	A	Agriculture	c_0	Total incon	20341	DOLLARS(millions)							
15	2011	A	Agriculture	c_0	Total exper	5914	DOLLARS(millions)							
16	2011	A	Agriculture	c_0	Total asset	43419	DOLLARS(millions)							
17	2011	A	Agriculture	j_Grand_Tc	Activity uni	27578	COUNT							
18	2011	A	Agriculture	j_Grand_Tc	Sales, gove	20112	COUNT							
19	2011	A	Agriculture	j_Grand_Tc	Total incon	24623	DOLLARS(millions)							
20	2011	A	Agriculture	j_Grand_Tc	Total exper	26523	DOLLARS(millions)							
21	2011	A	Agriculture	j_Grand_Tc	Total asset	20454	DOLLARS(millions)							
22	2011	B	Mining	a_0	Activity uni	35807	COUNT							
23	2011	B	Mining	a_0	Sales, gove	35601	COUNT							
24	2011	B	Mining	a_0	Total incon	46612	DOLLARS(millions)							
25	2011	B	Mining	a_0	Total exper	35804	DOLLARS(millions)							
26	2011	B	Mining	a_0	Total asset	6253	DOLLARS(millions)							

AUDIO FILE SCREENSHOT:

